



**UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CAMPUS APUCARANA
ENGENHARIA DE COMPUTAÇÃO
PROGRAMAÇÃO ORIENTADA A OBJETOS - C++17**

**QUEISE ÉLEN SANTOS SANTANA
RA: A2207257**

**CODEQUEST: A LENDA DO KERNEL
Sistema de Gerenciamento de Jogo de RPG
Relatório Técnico**

**APUCARANA
2026**



QUEISE ÉLEN SANTOS SANTANA
RA: A2207257

CODEQUEST: A LENDA DO KERNEL
Sistema de Gerenciamento de Jogo de RPG

Relatório técnico apresentado à disciplina de Programação Orientada a Objetos, do curso de Engenharia de Computação da Universidade Tecnológica Federal do Paraná - Campus Apucarana, como requisito avaliativo do projeto desenvolvido em C++17.

Professor(a): Prof. Dr. Rafael Gomes Mantovani

APUCARANA
2026

SUMÁRIO

1 Introdução.....	4
2 Arquitetura Geral.....	4
2.1 Diagrama de Classes.....	5
3 Decisões de Design.....	9
3.1 Composição em vez de herança para Inventário e Habilidades.....	9
3.2 Dependência circular entre Jogador e Inventario/Habilidade.....	9
3.3 Polimorfismo para custo de recursos, sem RTTI.....	9
3.4 Sobrecarga de operadores integrada à persistência.....	9
3.5 Reconstrução polimórfica na Persistência.....	10
3.6 Persistência completa do estado do jogo.....	10
3.7 Redistribuição de pontos via método virtual (RF6).....	10
3.8 Sistema de economia, loja e ItemEspecial.....	11
3.9 Separação entre menu interativo e demonstração automática (RF8).....	11
4 Padrões de Projeto Aplicados.....	11
4.1 Factory Method.....	11
4.2 Strategy.....	12
4.3 Observer.....	12
4.4 Singleton.....	12
5 Tratamento de Exceções.....	12
6 Testes e Cobertura.....	13
6.1 Documentação (Doxygen).....	14
7 Conclusão.....	14

1. Introdução

Este relatório descreve as decisões de design tomadas no desenvolvimento do CodeQuest: A Lenda do Kernel, um sistema de gerenciamento de jogo de RPG implementado em C++17 como projeto da disciplina de Programação Orientada a Objetos. O objetivo central do projeto foi aplicar, de forma integrada e justificada, os principais pilares da POO (encapsulamento, herança, polimorfismo e abstração), além de padrões de projeto clássicos, tratamento de exceções e organização modular de código.

O sistema simula um jogo de RPG por linha de comando: criação de personagens de diferentes classes com distribuição de pontos de atributo, gerenciamento de inventário com equipamentos, sistema de economia com moedas de ouro, loja para compra e venda de itens, itens especiais, um sistema de habilidades ativas e passivas, batalhas por turnos (bot vs bot, humano vs bot ou humano vs humano), level up com redistribuição de pontos e persistência completa do estado do jogo em arquivo - tudo acessível através de um menu interativo que cobre a totalidade dos requisitos funcionais da especificação.

2. Arquitetura Geral

Todo o código está organizado dentro do namespace rpg, dividido em módulos de responsabilidade única, cada um com seu par de arquivos .h/.cpp. Essa separação facilita a manutenção, reduz o tempo de recompilação e deixa explícitas as dependências entre as partes do sistema. A versão final inclui também o módulo de raças, o sistema de ouro e a loja integrada ao gerenciamento de inventário.

Quadro 1 - Organização modular do código

Módulo	Conteúdo
item.h / item.cpp	Item (abstrata), Arma, Armadura, Poção, ItemEspecial e produtos concretos
inventario.h / .cpp	Inventario (composição), equipamentos, uso de poções/itens especiais e exceções de capacidade/peso
habilidade.h / .cpp	Habilidade (abstrata) e as 5 habilidades concretas
observer.h	Padrão Observer (ObservadorDeEventos, NotificadorConsole)
raca.h / raca.cpp	Raça (abstrata) e raças concretas: Humano, Elfo, Anão e Orc
jogador.h / .cpp	Jogador - classe abstrata base, com HP, XP,

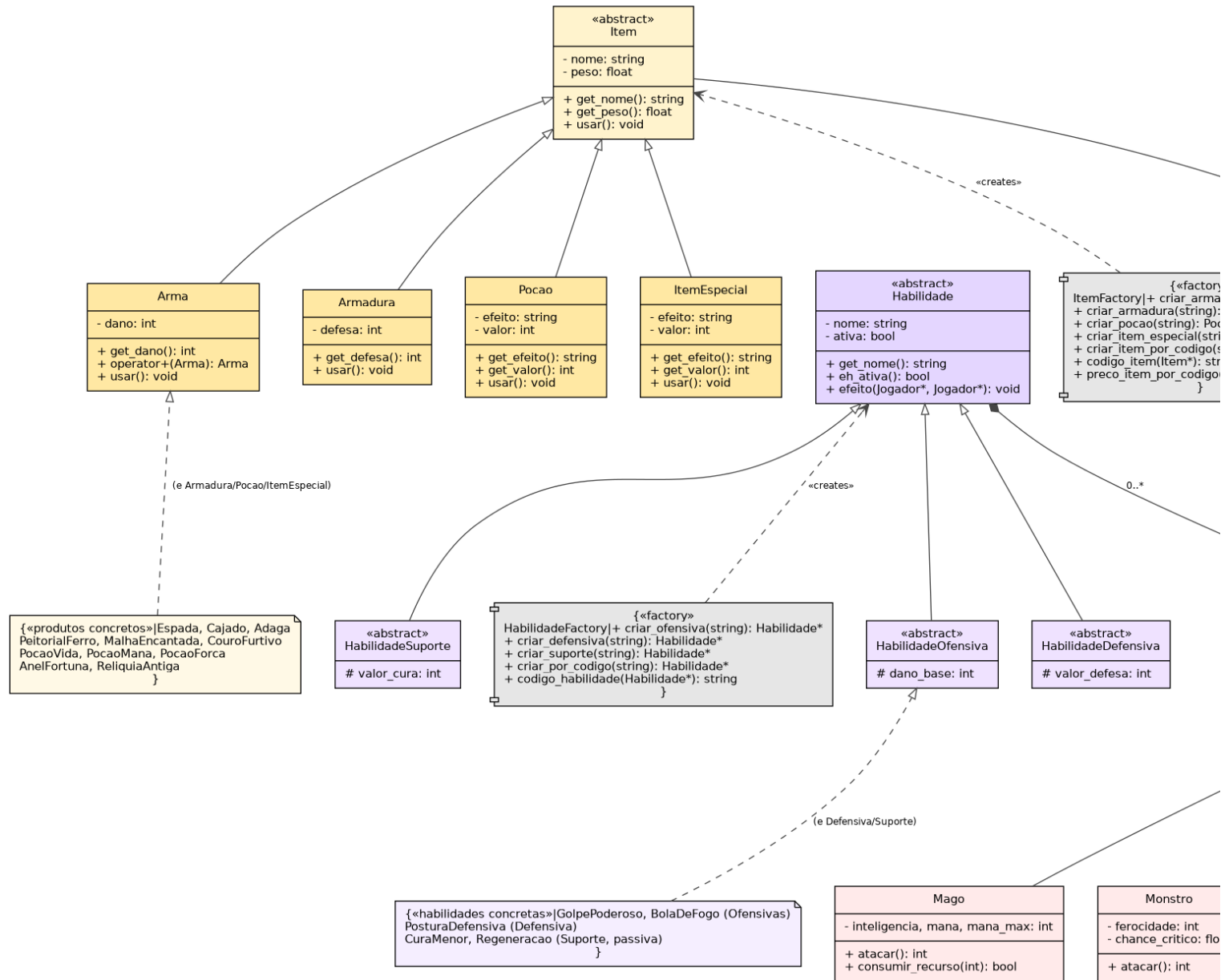
	ouro, raça, inventário e habilidades
personagens.h / .cpp	Classes concretas: Guerreiro, Mago, Monstro, Arqueiro e Clérigo
estrategia.h / .cpp	Padrão Strategy para escolha de alvo em batalha
factories.h / .cpp	ItemFactory, JogadorFactory, HabilidadeFactory e RacaFactory; catálogo e preços de itens
gerenciador.h / .cpp	GerenciadorDeJogo - Singleton com a coleção de personagens (RF1/RF2)
persistencia.h / .cpp	Salvar/carregar estado completo: personagens, raças, ouro, inventário, equipamentos e habilidades
batalha.h / .cpp	Sistema de combate por turnos com recompensa de XP e ouro ao vencedor
menu.h / .cpp	Menu interativo cobrindo RF1-RF8, loja, compra/venda de itens e listagem de ouro
main.cpp	Ponto de entrada: menu interativo ou demonstração automática (--demo)

Fonte: Elaborado pelo autor.

2.1 Diagrama de Classes

A figura a seguir resume a hierarquia de classes e os principais relacionamentos do sistema. Setas com ponta vazia indicam herança; losangos indicam composição; linhas tracejadas indicam dependência, como a criação de objetos pelas fábricas. O diagrama foi atualizado para contemplar ItemEspecial, o atributo de ouro no jogador, o catálogo de preços da loja e as fábricas de itens, personagens, habilidades e raças. Devido ao número de classes, a figura foi dividida em três partes, em páginas próprias, no formato paisagem, para melhor legibilidade.

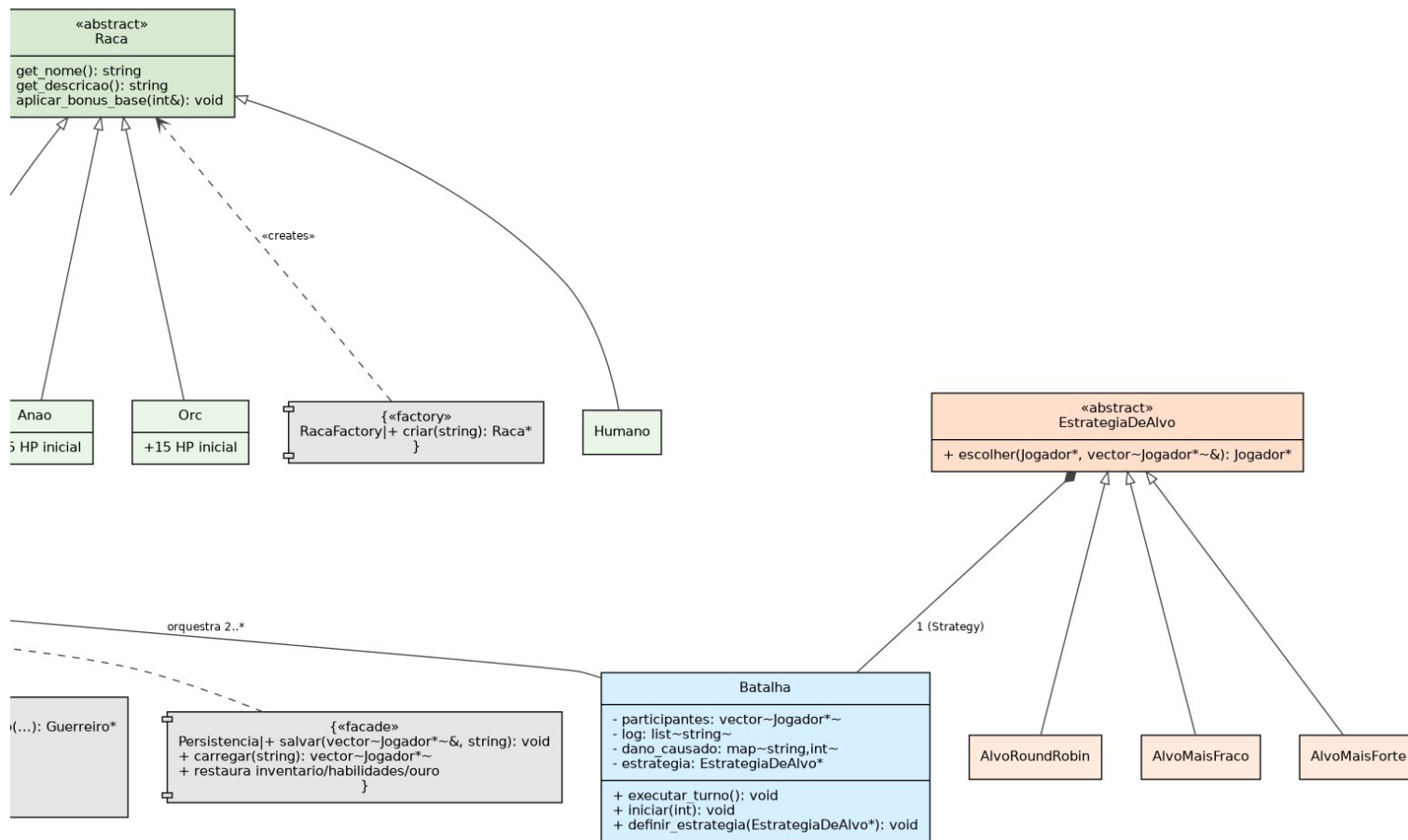
Figura 1 - Diagrama de classes do CodeQuest: A Lenda do Kernel (parte 1)



Fonte: Elaborado pelo autor a partir de docs/diagrama_classes.dot.

CodeQuest: A Lenda do Kernel — Diagrama de Classes
(setas vazias = heranca | losango = composicao | tracejado = dependencia)

Figura 1 - Diagrama de classes do CodeQuest: A Lenda do Kernel (parte 3)



Fonte: Elaborado pelo autor a partir de docs/diagrama_classes.dot.

3. Decisões de Design

3.1 Composição em vez de herança para Inventário e Habilidades

Optou-se por modelar a relação entre Jogador e Inventario como composição, uma vez que o Jogador possui um Inventario por valor, em vez de fazer o Jogador herdar funcionalidades de inventário diretamente. Essa escolha respeita o princípio de que herança representa uma relação "é um", enquanto composição representa "tem um": um Jogador não é um Inventário, ele possui um. O mesmo raciocínio se aplica à lista de habilidades aprendidas (vector<Habilidade*>).

3.2 Dependência circular entre Jogador e Inventario/Habilidade

Como o Inventario precisa chamar métodos do Jogador completo, por exemplo `dono->curar()` ao usar uma poção, e o Jogador precisa armazenar um Inventario por valor, exigindo o tipo completo no momento da declaração, surge uma dependência circular entre os dois módulos. A solução adotada segue o padrão clássico do C++: `inventario.h` apenas declara `class Jogador;` (forward declaration, suficiente para ponteiros e referências em assinaturas de método), enquanto `inventario.cpp` inclui `jogador.h` para implementar os métodos que de fato precisam do Jogador completo. O mesmo padrão foi aplicado entre Habilidade e Jogador, e entre `EstrategiaDeAlvo` e Jogador.

3.3 Polimorfismo para custo de recursos, sem RTTI

A habilidade Bola de Fogo consome mana, mas apenas a classe Mago possui esse atributo. Em vez de usar `dynamic_cast<Mago*>` dentro da habilidade, o que acoplaria a habilidade a uma subclasse específica, foi introduzido um método virtual genérico em Jogador: `virtual bool consumir_recurso(int)`, que por padrão sempre retorna verdadeiro, sem custo. O Mago sobrescreve esse método para de fato consumir mana. Assim, a habilidade chama `usuario->consumir_recurso(20)` sem saber ou se importar com o tipo real do personagem - um exemplo de polimorfismo bem aplicado no lugar de verificação de tipo em tempo de execução.

3.4 Sobrecarga de operadores integrada à persistência

O operador `operator<<` de Jogador não imprime o status completo, pois essa responsabilidade pertence ao método `exibirStatus()`, mais rico e voltado à leitura humana. Em vez disso, ele delega para o método virtual `serializar()`, que produz um formato compacto

separado por ponto e vírgula. Essa decisão permite que o mesmo operador seja reutilizado tanto para depuração no console (`cout << *jogador`) quanto para gravação em arquivo (`arquivo << *jogador`) pela classe `Persistencia`, evitando duplicar a lógica de formatação.

Foram também sobrecarregados `operator==` e `operator<` em `Jogador`, para comparação por nome e por HP atual, respectivamente, e `operator+` em `Arma`, que combina duas armas somando dano e peso - uma forma simplificada de crafting.

3.5 Reconstrução polimórfica na Persistência

Como cada subclasse de `Jogador` possui atributos próprios, como força, inteligência/mana, ferocidade/crítico, precisão e fé, a serialização é polimórfica: o método `serializar()` é virtual e cada subclasse o sobrescreve, prefixando a linha salva com o nome da própria classe, como "Guerreiro;..." ou "Monstro;...". Ao carregar, a classe `Persistencia` lê esse prefixo para decidir qual construtor concreto chamar, e em seguida usa `restaurar_estado()` para corrigir nível, HP, XP, atributos, inventário, equipamentos e habilidades para os valores exatos salvos, pois os construtores normais sempre inicializam um personagem novo.

3.6 Persistência completa do estado do jogo

A persistência foi ajustada para salvar e carregar o estado completo do jogo exigido pela especificação: dados do personagem, raça, classe concreta, nível, HP, XP, quantidade de ouro, atributos específicos, inventário, arma equipada, armadura equipada e habilidades aprendidas. Com isso, ao carregar um arquivo salvo, o sistema reconstrói os objetos por meio das fábricas correspondentes e restaura a sessão de jogo sem perder os elementos principais associados ao personagem, inclusive seu saldo econômico.

3.7 Redistribuição de pontos via método virtual (RF6)

Ao subir de nível, o personagem recebe pontos extras para distribuir em seu atributo principal. Em vez de o menu, camada de interface, precisar saber qual atributo pertence a qual classe, foi adicionado virtual `void redistribuir_pontos(int)` em `Jogador` - implementação padrão vazia, sobrescrita por `Guerreiro` (soma à força), `Mago` (à inteligência), `Monstro` (à ferocidade), `Arqueiro` (à precisão) e `Clérigo` (à fé). O menu apenas detecta que o nível mudou após `ganhar_xp()` e chama `p->redistribuir_pontos(pontos)` polimorficamente, sem nenhum `if/else` por tipo de classe.

3.8 Sistema de economia, loja e ItemEspecial

Para tornar a compra de itens coerente com as regras do jogo, foi adicionado o atributo ouro em Jogador. Cada personagem inicia com saldo próprio e possui métodos `get_ouro()`, `ganhar_ouro()`, `gastar_ouro()` e `restaurar_ouro()`, usados pela loja, pelas recompensas de batalha e pela persistência. O menu de inventário passou a funcionar também como uma loja: o jogador visualiza seu ouro disponível, consulta os preços, compra itens apenas se tiver saldo suficiente e pode vender/remover itens recebendo metade do preço de catálogo. Caso a compra falhe por limite de peso, capacidade ou item inexistente, o valor é estornado automaticamente.

Também foi criada a classe `ItemEspecial`, derivada de `Item`, com efeito e valor próprios. Os produtos concretos `AnelFortuna` e `ReliquiaAntiga` podem ser criados pela `ItemFactory`, usados pelo `Inventario` e serializados/carregados pela `Persistencia`. O `AnelFortuna` reforça a economia ao conceder ouro ao personagem, enquanto a `ReliquiaAntiga` representa um item especial de catálogo com preço próprio. Além disso, a listagem geral de personagens foi atualizada para exibir o saldo de ouro de cada personagem junto das informações principais.

3.9 Separação entre menu interativo e demonstração automática (RF8)

O executável aceita um argumento opcional: sem argumentos, abre o menu interativo completo (`executar_menu_interativo()`, em `menu.cpp`), cobrindo todas as operações da especificação através de `std::cin`. Com `--demo`, executa um roteiro automático que não depende de entrada do usuário - útil para verificação rápida de que todos os módulos continuam funcionando após uma alteração, sem precisar digitar dezenas de opções manualmente. Um cuidado de robustez tratado explicitamente no menu é que, ao detectar EOF na entrada padrão, seja por arquivo redirecionado terminado ou Ctrl+D no terminal, uma exceção de controle de fluxo (`EntradaFinalizadaException`) interrompe o loop principal de forma limpa, evitando um laço infinito de "entrada inválida".

4. Padrões de Projeto Aplicados

4.1 Factory Method

Quatro fábricas (`ItemFactory`, `JogadorFactory`, `HabilidadeFactory` e `RacaFactory`) centralizam a criação de objetos a partir de uma string de tipo, evitando que o código cliente precise conhecer as classes concretas ou seus construtores. A `ItemFactory` também concentra

a codificação e o preço dos itens do catálogo, incluindo armas, armaduras, poções e itens especiais. Novos tipos de item, personagem, habilidade ou raça podem ser adicionados alterando apenas a fábrica correspondente.

4.2 Strategy

A lógica de "quem é o próximo alvo" em uma batalha foi extraída da classe Batalha para uma hierarquia de EstrategiaDeAlvo intercambiável (AlvoRoundRobin, AlvoMaisFraco, AlvoMaisForte). A Batalha delega a escolha para a estratégia configurada e expõe definir_estrategia() para trocar o algoritmo em tempo de execução, sem nenhuma alteração na classe Batalha - a exata motivação do padrão.

4.3 Observer

Jogador mantém uma lista de ObservadorDeEventos* e dispara notificações quando sobe de nível ou é derrotado, sem precisar conhecer quem está observando. Isso desacopla a lógica de jogo, em Jogador::receber_dano() e subir_nivel(), de qualquer mecanismo de notificação específico. O NotificadorConsole fornecido apenas imprime no terminal, mas poderia ser substituído por um log em arquivo ou uma interface gráfica sem alterar uma linha de Jogador.

4.4 Singleton

GerenciadorDeJogo garante que exista exatamente uma coleção de personagens durante toda a execução do programa, acessível globalmente via GerenciadorDeJogo::instance(). O construtor é privado e a cópia é explicitamente proibida (= delete), forçando todo o código - em particular o menu interativo - a operar sobre a mesma lista de personagens. Um método destruir_instancia() permite encerrar a sessão liberando a memória de todos os personagens e recomeçar do zero, o que também é usado para isolar os testes unitários do Singleton entre si.

5. Tratamento de Exceções

Cada módulo que representa uma regra de negócio possui suas próprias exceções customizadas, todas derivadas de std::runtime_error ou std::invalid_argument nas fábricas:

Quadro 2 - Exceções customizadas por módulo

Módulo	Exceções
Inventario	InventarioCheioError, PesoExcedidoError, ItemNaoEncontradoError

Habilidade	HabilidadeNaoEncontradaError, HabilidadePassivaError
Persistencia	PersistenciaError
Factories	std::invalid_argument (tipo inexistente no catálogo)

Fonte: Elaborado pelo autor.

Todas as exceções são lançadas o mais próximo possível da violação da regra, por exemplo dentro de `Inventario::adicionar_item()` antes de qualquer efeito colateral, e capturadas nos pontos de uso em `main.cpp` e na classe `Batalha`, que precisa continuar a partida mesmo que uma ação específica falhe.

6. Testes e Cobertura

Os testes unitários usam Google Test e estão organizados em `tests/test_rpg.cpp`, cobrindo: encapsulamento, herança e polimorfismo (dano diferente por classe, coleção heterogênea), classes abstratas, exceções customizadas, o módulo de inventário (equipar, desequipar, peso excedido, uso de poções), os 4 padrões de projeto (Factory, Strategy, Observer e Singleton), a redistribuição de pontos no level up, a sobrecarga de operadores e o ciclo completo de salvar/carregar do módulo de persistência. No total, 40 casos de teste são executados via CMake/CTest, todos passando. O modo `--demo` também demonstra o catálogo com preços, o uso de `ItemEspecial`, o saldo de ouro e a listagem de personagens com ouro.

A cobertura registrada foi medida com `gcov/lcov`, considerando a execução automática dos testes unitários, sem contar a execução manual do menu interativo ou do modo `--demo`, que elevaria a cobertura combinada para 85,9%:

Tabela 1 - Resultado da cobertura de testes

Métrica	Resultado	Mínimo exigido
Linhas cobertas	62,4% (396 de 635)	60%
Funções cobertas	70,8% (119 de 168)	-

Fonte: Elaborado pelo autor.

O relatório HTML detalhado por arquivo está em `docs/coverage/index.html`. Os módulos com maior cobertura são os mais ricos em regras de negócio testáveis isoladamente, como `Item`, `Habilidade` e `Estrategia`. Os com menor cobertura, como `Batalha` e `Persistencia`, têm parte de sua lógica em loops e fluxos de E/S exercitados mais naturalmente por teste de integração, no modo `--demo`, do que por testes unitários isolados.

6.1 Documentação (Doxygen)

Todas as classes possuem comentários no formato Doxygen (`/** @brief ... */`), e o projeto inclui um Doxyfile configurado. Rodando `doxygen Doxyfile` a partir da raiz do projeto, gera-se documentação HTML navegável em `docs/doxygen/html/index.html`,

incluindo diagramas de herança e de colaboração entre classes gerados automaticamente pelo Graphviz. O README também foi atualizado para descrever o ItemEspecial, a moeda de ouro, a loja e os comandos de execução.

7. Conclusão

O CodeQuest: A Lenda do Kernel evoluiu de um protótipo de console para um sistema modular organizado em namespace, com quatorze módulos de responsabilidade única, quatro padrões de projeto aplicados de forma justificada (Factory, Strategy, Observer e Singleton - não apenas "para cumprir requisito"), tratamento de exceções consistente, sobrecarga de operadores integrada à persistência, um menu interativo cobrindo a totalidade dos requisitos funcionais, sistema de economia com ouro, loja de compra/venda e uma suíte de 40 testes automatizados com 62,4% de cobertura de linhas. As decisões documentadas neste relatório priorizaram, sempre que possível, soluções polimórficas em vez de verificações de tipo em tempo de execução, e desacoplamento entre módulos por meio de forward declarations e injeção de estratégias/observadores.

Com os ajustes finais, a persistência passou a contemplar também inventário, equipamentos, habilidades e ouro, enquanto a hierarquia de personagens foi ampliada para incluir Monstro, Arqueiro e Clérigo, além das raças concretas Humano, Elfo, Anão e Orc. A inclusão de ItemEspecial, AnelFortuna, ReliquiaAntiga, catálogo de preços, compra/venda com validação de saldo e exibição de ouro na listagem de personagens deixa o projeto mais alinhado à especificação da disciplina e demonstra de maneira mais completa os conceitos centrais de Programação Orientada a Objetos exigidos.